

SYSTEM ANALYST

presented by Ayub

Vibecoding

Что сегодня

- AI-агенты и инструменты: что это и что открывать
- Как **найти и проверить идею**, за которую платят
- Как **правильно промптить**, чтобы AI понял с первого раза
- Грабли: утечка секретов, взлом, оверинжиниринг
- База: Git, тесты, Docker, хостинг
- Как **прокачать AI**: свои инструкции (SKILL.md) и данные (MCP)
- Практика: собираем приложение с нуля

**Хакатон Anthropic. В зале инженеры против людей,
которые вообще не умеют кодить.**

Угадай: кто занял первые места? И почему?

Победили не программисты

- Хакатон Anthropic: **13 000** заявок, взяли **500**, почти все разработчики
- **1 место: юрист**. Сделал AI-инструмент для проверки разрешений на стройку
- **3 место: кардиолог**
- **4 из 5** победителей не были профессиональными разработчиками

"Я выиграл этот конкурс и не написал ни строчки кода." Майк Браун, юрист, 1 место

Почему так вышло

- Проблема **никогда не была в коде**. Она всегда в том, ЧТО решать
- Программист умеет писать код, но часто не знает, какую боль лечить
- Эксперт знает боль насквозь, но раньше не мог построить: нанять разработчика дорого
- AI убрал этот барьер. Теперь эксперт строит решение сам

Юрист и врач знали реальную проблему. AI дал им руки. Поэтому они и выиграли.

Что такое vibecoding

- Ты описываешь словами, **что хочешь**
- AI пишет код за тебя
- Ты смотришь результат, говоришь что не так, AI правит
- Повторяешь, пока не работает

Vibecoding: ты ведёшь по смыслу (vibe), код пишет машина.

Ты пишешь в ChatGPT "сделай мне сайт". Он выдаёт текст с кодом.

Кто этот код запустит, проверит и зальёт на сервер?

AI-чат против AI-агента

	AI-чат	AI-агент
Что делает	отвечает текстом	выполняет работу
Действия	ты копируешь руками	сам жмёт, ходит по шагам
Инструменты	нет	файлы, терминал, браузер
Память о задаче	в рамках сообщения	держит цель до конца

Чат говорит **как** сделать. Агент **берёт и делает**.

Чем агент реально полезен

- Сам читает твои файлы и понимает проект
- Сам запускает команды и видит ошибки
- Сам исправляет и проверяет результат
- Доводит задачу до конца, а не до совета

Чат это консультант по телефону. Агент это мастер, который пришёл и починил.

Инструменты: что открывать

- **AI-чаты** (ChatGPT, Claude): спрашиваешь, код копируешь руками
- **Агенты в редакторе** (Cursor, Claude Code, Windsurf, Copilot): сами правят файлы и запускают команды
- **Сборщики приложений** (Lovable, Bolt, v0, Replit): описал приложение, получил готовое и сразу в сети

Новичку начинать со **сборщика** (Lovable или Bolt): всё под капотом, кода видеть не надо.

AI это твоя дешёвая команда 24/7

- **Ассистент**: берёт рутину, пока ты думаешь
- **Репетитор**: объясняет любую тему столько раз, сколько надо
- **Ревьюер кода**: смотрит твой код и показывает, где слабо
- **Senior-инженер**: учит писать лучше, по одним стандартам

Нанять всех четверых дорого. Одна подписка на AI = все они, 24/7. Отговорок не учиться больше нет.

Module Check 1

1. Чем **агент** отличается от **чата** своими словами?
2. ChatGPT написал тебе код. Почему это ещё не готовый продукт?
3. Что значит **vibecoding**?
4. С какого типа инструмента новичку проще начать и почему?

Ты садишься за стол и пытаешься придумать гениальную идею стартапа с нуля.

Почему так почти всегда выходит пустышка?

Хорошие идеи не выдумывают, их замечают

- Выдуманные за столом идеи (PG зовёт их "ситком-идеи") звучат складно, но юзеров не дают
- Сильные идеи приходят, когда ты **живёшь с проблемой** и видишь, чего не хватает
- Лучшая идея = ту, что ты сам хочешь, можешь сделать, а другие не замечают

"Лучший способ придумать идею стартапа: не пытаться её придумать." Пол Грэм

5 источников хорошей идеи

- **Своя боль**: реши проблему, которая есть у тебя самого
- **Твоя сила**: что ты знаешь или умеешь лучше других (прошлая работа, хобби)
- **Грязная работа**: сложное, что все обходят стороной (там меньше конкурентов)
- **Жизнь в будущем**: тебя бесит то, чего ещё нет, потому что ты уже видишь дальше
- **Сначала боль, потом AI**: не бери технологию и не ищи ей применение

| Главная ошибка новичков: "хочу что-то на AI". Это решение в поиске проблемы.

Хорошая идея часто выглядит плохо

- Если бы идея явно выглядела отличной, её бы уже кто-то делал
- Значит хорошая идея должна казаться немного безумной
- Целься в **узкую группу**, которой нужно отчаянно, а не в большую, которой так себе
- Facebook начинался только для Гарварда, потом расширился

Лучше 100 человек, которые тебя обожают, чем 10 000 равнодушных.

Стоит ли делать: быстрый чек

- Можешь назвать конкретных людей, кому нужна даже кривая версия 1?
- Ты сам этого хочешь?
- Это стало возможно только недавно?
- У тебя есть реальный новый взгляд на проблему?

Не можешь назвать ни одного отчаянного пользователя = идея пока слабая.

Ты показал идею другу, маме, AI-чату. Все сказали "круто, делай!".

Можно начинать строить?

Главная ловушка: все тебя хвалят

- **Друзья и близкие** хвалят, чтобы не обидеть. Это вежливость, не данные
- **AI-чат хвалит ещё сильнее**: он обучен быть приятным и соглашаться
- Реальный случай: ChatGPT назвал идею "бизнес на палке с какашкой" гениальной
- Похвала AI это не валидация. Это его характер

Сначала кажется, что у тебя гениальная идея. На деле тебе просто поддакнули.

Преврати AI из подлизы в критика

- По умолчанию AI это да-человек. Надо **приказать** ему спорить с тобой
- Промпт: "Будь скептическим инвестором. Разнеси мою идею"
- Проси: 5 главных рисков, самые дорогие допущения, красные флаги, почему бы ты отказал
- Даже после этого настоящая проверка не у AI, а у живых людей

"Act as a skeptical investor. Challenge my idea, list the 5 biggest risks and why you'd pass."

The Mom Test: как говорить с людьми

- Книга Роба Фицпатрика про то, как узнать правду, а не комплименты
- 3 правила:
 - Говори про **их жизнь**, а не про твою идею
 - Спрашивай про **конкретное прошлое**, не про гипотетическое будущее
 - **Меньше говори, больше слушай**

Нельзя спрашивать "нравится ли тебе моя идея" вообще никого.

Плохие вопросы против хороших

Плохой вопрос	Хороший вопрос
Тебе нравится идея?	Расскажи, как ты решаешь это сейчас
Купил бы такое?	Когда ты последний раз с этим столкнулся?
Ты бы платил \$10?	Сколько тебе уже стоит эта проблема?

Как проверить идею по-настоящему

- **Поговори с людьми** по правилам Mom Test, ищи факты и боль
- **Лендинг + список ожидания**: страница "скоро", собери почты, немного рекламы
- **Предпродажа**: человек, который заплатил вперёд, это сильнейший сигнал
- **Делай то, что не масштабируется**: первых юзеров находи руками

Стартап killer номер 1: построил то, что никому не нужно (~4 из 10 провалов).

Книги, которые стоит прочитать

Книга	О чём
The Mom Test	как говорить с клиентами, чтобы не врали. Читать первой
The Lean Startup	MVP, цикл собери-измерь-научись, проверяй гипотезы
Zero to One	создавай новое (0 в 1), а не копируй. Питер Тиль

Module Check 2

1. Почему **AI хвалит** почти любую идею?
2. Каким промптом превратить AI в **критика**?
3. Назови 1 правило **Mom Test**.
4. Почему **предоплата** клиента важнее, чем его "да, куплю"?

**Парень за выходные собрал AI-приложение. Выложил
ключ от платного сервиса прямо в код.
Через 3 дня пришёл счёт на 12 000 долларов. Как так
вышло?**

Грабли 1: утечка секретов

- **Секрет (API-ключ, токен, пароль)** = ключ от твоего кошелька
- Зашил ключ в код и выложил в открытый доступ = отдал всем
- Боты сканируют GitHub и находят ключи за минуты
- За год на публичном GitHub утекло больше 10 миллионов секретов

Секреты живут в `.env`, а `.env` **никогда** не уходит в публичный доступ.

Грабли 2: взлом и пустой счёт

- Открытая база без пароля = заходи кто хочешь
- Нет лимита трат = один злоумышленник разорит
- Слепое доверие коду от AI = дыры, которых ты не видишь
- Аудит 1645 AI-приложений: ~10% имели критические дыры

| Всегда ставь **лимит трат** на платных сервисах. Это страховка от банкротства.

Чек-лист безопасности

- Секреты в `.env` , не в коде
- Лимит трат на каждом платном сервисе
- Пароль и доступ на базу данных
- Не выкладывай рабочие ключи в публичный репозиторий
- Спрашивай AI прямо: "проверь на дыры в безопасности"

Module Check 3

1. Почему **нельзя** оставлять ключ прямо в коде?
2. Что такое **лимит трат** и от чего он спасает?
3. Назови 3 пункта из чек-листа безопасности.

**Тебе надо простую форму обратной связи. Ты строишь
микросервисы, очереди и три базы данных.
Что пошло не так?**

Грабли 3: оверинжиниринг

- **Оверинжиниринг** = делаешь сложнее, чем нужно задаче
- Тратишь недели там, где хватило бы вечера
- Чем больше деталей, тем больше ломается
- Выжигашь мозг и бросаешь проект

Сначала самое простое, что работает. Усложняй только когда реально упёрся.

Как делать эффективно

- Опиши задачу **одним предложением**: какую боль решаем
- Сделай минимальную рабочую версию
- Покажи людям, собери реакцию
- Усложняй только то, что реально просят

Рабочее и простое всегда бьёт идеальное и недоделанное.

Где vibecoding силён, а где опасен

Силён	Опасен
MVP и прототипы	платежи и деньги
лендинги	мед, финансы, регуляторика
внутренние инструменты	высокие нагрузки и масштаб
простые CRUD-приложения	большой готовый код

Правило: вайбкодишь прототип, потом добавляешь настоящую инженерию перед продакшеном.

Проблема последних процентов

- AI быстро даёт ~70%: каркас, очевидные куски
- Последние 30% (крайние случаи, отладка, прод, безопасность) так же сложны, как всегда
- "Выглядит готово" и "реально готово к продакшену" это две разные точки
- Застрять на последней миле это норма, а не твой провал

"AI доведёт почти до решения, но не до конца. Выглядит рабочим, а внутри держится на скотче." Эдди Османи

Сколько это стоит

- Подписка тула обычно **~\$10-50 в месяц** (оценка)
- Плюс расход кредитов и токенов: чем больше итераций, тем больше счёт
- Агенты жгут в **10-100 раз** больше токенов, чем чат: контекст шлётся каждый шаг
- Часть денег уходит на то, что AI сам крутится в отладке

Дешевле, чем нанимать разработчиков. Но не бесплатно. Следи за счётчиком и ставь лимиты.

**Ты сказал AI "сделай приложение для заметок". Он
сделал. Но не то, что ты представлял.
Чего не хватило в задаче?**

Здесь работает то, что мы уже знаем

- AI это разработчик. Дай ему **нормальное ТЗ**
- Вспоминаем артефакты аналитика:
 - **BR** (бизнес-требования): зачем вообще
 - **SRS**: что система делает
 - **FR** (функциональные): что умеет
 - **NFR** (нефункциональные): как быстро, надёжно, безопасно

Сейчас это называют **spec-driven development** (разработка через спецификацию).

Почему спецификация решает всё

- Размытая просьба = размытый результат
- Чёткие FR = AI знает что строить
- NFR = AI знает про скорость, лимиты, безопасность
- Спека = твой контроль над AI

Качество промпта = качество продукта. Аналитик тут в выигрыше.

Диаграммы тоже генерит AI

- **UML**: как устроена система внутри (классы, связи)
- **BPMN**: как идёт процесс по шагам
- Описал словами = AI нарисовал схему
- Проверил, поправил, вставил в документацию

Аналитик + AI = спека и диаграммы за минуты, а не за дни.

**Два человека просят AI одно и то же. Один получает мусор, другой рабочий код.
В чём разница?**

Анатомия хорошего промпта

- **Роль:** кем быть ("ты senior backend-инженер")
- **Задача:** что именно сделать, одним чётким запросом
- **Контекст:** твой проект, стек, ограничения (AI этого не знает)
- **Пример:** покажи, как выглядит "хорошо" (few-shot)
- **Формат:** как оформить ответ (файл, блок кода, по шагам)
- **Ограничения:** чего избегать, какие библиотеки, крайние случаи

AI не телепат. Он строит то, что ты описал, а не то, что ты представил.

Техники, которые работают

- **Будь конкретным**: точные требования вместо размытых просьб
- **Малыми шагами**: большую фичу руби на маленькие промпты
- **Chain-of-thought**: проси "думай по шагам", AI ловит больше ошибок в логике
- **Few-shot**: покажи пример, а не только опиши словами
- **Верни ошибку назад**: вставь текст ошибки И код, AI сам починит
- **Итерируй**: сначала простая рабочая версия, потом улучшай
- Говори, **что делать**, а не чего не делать

Плохой против хорошего промпта

Плохой	Хороший
"Сделай страницу логина"	"Ты frontend-разработчик. React, task-app. Логин: поля email и пароль, кнопка Войти, красная ошибка если поля пустые. Чистый CSS, без UI-библиотек"
"Сломалось, почини"	"При сабмите ошибка TypeError ... id of undefined, строка 42 api.js. Сначала объясни причину, потом исправленный код. [код + ошибка]"

Формула: Роль + Задача + Контекст + Ограничения (плюс пример или ошибка).

Когда AI застрял в цикле

- После **~5 неудачных попыток** остановись, не долби тот же промпт
- **Вернись к последней рабочей версии** и опиши заново
- **Свежий чат**: новый контекст заставит AI перечитать код и зайти иначе
- **Упрости**: разбей на меньшие шаги, одна фича за раз
- **Смени модель или тул**, если один упорно фейлит одинаково

Цикл провалов жрёт деньги и нервы. Стоп, откат, свежий заход.

AI пишет за тебя весь код. Зачем тебе вообще что-то знать про Git, серверы и Docker?

База, которую всё равно надо знать

- AI печатает код, но **рулишь ты**. База это твои рычаги
- Без неё ты не можешь: откатить плохое изменение, залить приложение в интернет, понять что сломалось
- Не надо уметь писать всё руками. Надо **понимать, что происходит**
- Сначала **подстраховка** (Git, тесты), потом **выпуск** (Docker, домен, хостинг)

Без базы ты не управляешь AI, ты ему молишься.

Git: машина времени для проекта

- **Git** = система точек сохранения для всего проекта
- Записывает каждое изменение, можно откатиться на любой момент
- Как сейвы в видеоигре: AI сломал = откатился назад
- **GitHub** = облачный дом и бэкап твоего проекта

| *Сделал коммит перед каждым большим изменением = всегда можешь вернуться.*

**AI написал тебе 500 строк кода. Ты поменял одну деталь.
Как понять, что ты случайно не сломал что-то на другом
экране?**

Тесты: ремни безопасности кода

- **Тест** = код, который проверяет, что твой код работает правильно
- Без тестов каждое изменение это лотерея: вдруг тихо сломал что-то в другом месте
- Тесты ловят то, что "я кликнул, вроде работает" никогда не поймают
- Покрыл код тестами = меняешь спокойно, баги не уезжают в продакшен

Без тестов нет страховки. Каждое изменение это ставка вслепую.

Магия: AI сам чинит по тестам

- AI-агент пишет код, **сам запускает тесты**, видит что упало, чинит, запускает снова
- Цикл повторяется, пока тесты не станут зелёными
- Тест это сигнал агенту: правильно или нет. Нет тестов = агент работает вслепую
- AI и сами тесты напишет за тебя, но их надо **проверять** (бывают поверхностные)

Тесты это поводок для AI. С ними он сам себя исправляет, без них творит вслепую.

Правило: покрывай код тестами

- Не начинай со ста тестов. Начни с **одного**
- Один тест на самый важный сценарий (логин, оплата, отправка формы)
- Дальше добавляй по мере роста приложения
- Проси AI: "напиши тест на этот сценарий и прогони его"

Один настоящий тест бьёт сотню предположений "вроде работает".

Подстраховка есть: Git откатывает, тесты ловят.
Теперь как вынести приложение из ноутбука в интернет?

Docker: приложение в коробке

- **Docker-контейнер** = твоё приложение плюс всё, что ему нужно, в одной коробке
- Код, библиотеки, настройки упакованы вместе
- Работает одинаково где угодно: у тебя, у друга, на сервере
- Убивает классику "у меня на компе работало, а у вас нет"

Контейнер = коробка, которая ведёт себя одинаково, где её ни открой.

Домен и хостинг: вынести в интернет

- **Домен** = человеческий адрес сайта (myapp.com). Ты его арендуешь, не покупаешь навсегда
- **Хостинг** = всегда включённый компьютер, где живёт твоё приложение
- **Deploy (деплой)** = залить приложение туда, чтобы оно стало доступно людям
- Это прыжок от "работает у меня на компе" к "живёт в интернете"

Для старта: Vercel, Netlify, Railway. Залил за пару минут, и приложение в сети.

Module Check 4

1. Зачем нужен **Git** и чем он спасает в работе с AI?
2. Как **AI сам себя чинит** с помощью тестов?
3. Что такое **Docker-контейнер** простыми словами?
4. Что значит **задеплоить** приложение?

Ты пишешь AI одни и те же инструкции в каждом новом чате. Каждый раз заново.

Можно один раз и навсегда?

Запиши инструкции один раз

- Не повторяй один и тот же промпт. Положи его в файл, AI читает сам
- **CLAUDE.md**: правила проекта, AI читает в начале каждой сессии
- **AGENTS.md**: общий стандарт, один файл для разных тулов
- **SKILL.md**: навык под конкретную задачу, грузится когда нужен
- Чем детальнее файл, тем лучше и стабильнее результат

CLAUDE.md это устав сотрудника (всегда). SKILL.md это инструкция под одну задачу (по нужде).

МСР: подключи AI к своим данным

- По умолчанию AI знает только то, что ты вставил в чат
- **МСР** = USB-C для AI: один стандарт подключать AI к твоим тулам и данным
- Коннекторы: Obsidian, Jira, GitHub, Google Drive, Slack, база данных
- Утром говорю Claude Code "составь план": он читает мои планы в Obsidian, тянет активные тикеты из Jira и сам собирает план

Подключил один раз. Теперь AI работает с твоими реальными заметками, тикетами и кодом, а не с пустого листа.

Золотые правила вайбкодера

- **Коммить часто:** каждая рабочая версия это точка отката
- **Читай код AI:** это черновик, а не истина. Проверь
- **Малыми шагами:** не проси весь проект за один промпт
- **Человек решает про деньги и безопасность:** AI выберет "работает", а не "безопасно"
- **Не правь продакшен напрямую,** держи бэкапы
- **Знай, когда звать живого разработчика:** деньги, данные людей, чего не понимаешь

Время практики.

Собираем приложение с нуля. Ноль строчек кода. Только промпты.

Демо: план действий

1. Выбираем **узкую** идею с реальной болью
2. Быстро проверяем её: AI-критик плюс пара вопросов по Mom Test
3. Открываем **сборщик приложений** (Lovable или Bolt)
4. Пишем **BR** и **SRS** (FR + NFR) как ТЗ для AI
5. Генерим **UML** и **BPMN** с AI
6. Собираем приложение **хорошими промптами** (роль, задача, контекст)
7. Ставим **лимиты, прячем секреты, добавляем тест**, коммитим
8. Проверяем, правим словами, повторяем

Module Check: финал

1. Откуда берутся **хорошие идеи** и почему не из головы за столом?
2. Почему нельзя верить **похвале AI** и что с этим делать?
3. Из чего состоит **хороший промпт**?
4. Какие артефакты дают AI как **ТЗ**? (BR, SRS, FR, NFR)
5. Зачем нужны **Git** и **тесты** в работе с AI?
6. Что такое **МСР** и зачем подключать AI к своим данным?
7. Назови 2 правила **безопасности**.

Главное за сегодня

Идею не выдумывают, её замечают через свою боль. Узкая ниша.

AI хвалит всех. Проверяй идею у людей, не у чата.

Хороший промпт = роль, задача, контекст, пример. Детально решает.

Секреты в `.env`, **лимиты** везде. Иначе банкротство.

Спека (BR, SRS, FR, NFR) = твой контроль над AI.

Git и тесты = откат и страховка. AI сам чинит по тестам.

MCP и SKILL.md = подключи данные, запиши инструкции один раз.